

1. Vacuum cleaner

```
PS C:\Users\Administrator\Documents\Artificial-Intelligence---Lab> g++ "Vacuum cleaner.cpp" -o vacuum
PS C:\Users\Administrator\Documents\Artificial-Intelligence---Lab> .\vacuum.exe
Enter the Size of the Room: 3
Enter Room States (0-Clean, 1-Dirty):
1 0 1
0 1 0
1 0 1
Enter Starting Position: 1 1

===== Vacuum Cleaning Started =====

Vacuum moved to (1,1)
Dirty -> Cleaning...

Moving from (1,1) to (0,1)
Vacuum moved to (0,1)
Already Clean.

Moving from (0,1) to (0,0)
Vacuum moved to (0,0)
Dirty -> Cleaning...
```

Activate V

```
Moving from (0,0) to (1,0)
Vacuum moved to (1,0)
Already Clean.

Moving from (1,0) to (2,0)
Vacuum moved to (2,0)
Dirty -> Cleaning...

Moving from (2,0) to (2,1)
Vacuum moved to (2,1)
Already Clean.

Moving from (2,1) to (2,2)
Vacuum moved to (2,2)
Dirty -> Cleaning...

Moving from (2,2) to (1,2)
Vacuum moved to (1,2)
Already Clean.

Moving from (1,2) to (0,2)
Vacuum moved to (0,2)
```

```
Moving from (2,2) to (1,2)
Vacuum moved to (1,2)
Already Clean.
```

```
Moving from (1,2) to (0,2)
Vacuum moved to (0,2)
Dirty -> Cleaning...
```

```
===== Final Room State =====
```

```
0 0 0
```

```
0 0 0
```

```
0 0 0
```

```
PS C:\Users\Administrator\Documents\Artificial-Intelligence---Lab>
```

2a.

C:\Users\Administrator\Documents\2a.exe

```
Enter Initial State:
```

```
1 3 6
```

```
5 0 2
```

```
4 7 8
```

```
Enter Goal State:
```

```
1 2 3
```

```
4 5 6
```

```
7 8 0
```

```
Goal State Reached!
```

```
Step 0
```

```
Initial State
```

```
1 3 6
```

```
5 _ 2
```

```
4 7 8
```

```
-----
```

```
Step 1
```

```
Move : Right
```

```
1 3 6
```

```
5 2 _
```

```
4 7 8
```

```
-----
```

```
Step 2
```

```
Move : Up
```

```
1 3 _
```

```
5 2 6
```

```
4 7 8
```

```
-----
```

```
Step 3
```

```
Move : Left
```

```
1 _ 3
```

```
5 2 6
```

```
4 7 8
```

```
-----
```

```
Step 4
```

```
Move : Down
```

```
1 2 3
```

```
5 _ 6
```

```
4 7 8
```

```

-----
Step 5
Move : Left
1 2 3
_ 5 6
4 7 8
-----
Step 6
Move : Down
1 2 3
4 5 6
_ 7 8
-----
Step 7
Move : Right
1 2 3
4 5 6
7 _ 8
-----
Step 8
Move : Right
1 2 3
4 5 6
7 8 _
-----
Total Moves = 8

Process returned 0 (0x0)   execution time : 133.530 s
Press any key to continue.

```

2b.

 C:\Users\Administrator\Documents\2b.exe

```

Enter board size: 3

Knight's Tour Found:

1   4   7
6  XX  2
3   8   5

Process returned 0 (0x0)   execution time : 15.559 s
Press any key to continue.

```

2c.

```
Enter maze size: 5 5
Enter Start position: 0 0
Enter Treasure position: 4
4
Enter number of danger zones: 3
Enter danger zone positions:
1 1
2 2
3
3

Shortest Path:
(0,0) -> (0,1) -> (0,2) -> (1,2) -> (1,3) -> (2,3) -> (2,4) -> (3,4) -> (4,4)
Path Cost: 8
```

2d

```
C:\Users\Administrator\Documents\2d.exe
DFS Solution:
. Q . .
. . . Q
Q . . .
. . Q .

BFS Solution:
. Q . .
. . . Q
Q . . .
. . Q .

A* Solution:
. . Q .
Q . . .
. . . Q
. Q . .

Process returned 0 (0x0)   execution time : 0.105 s
Press any key to continue.
```

3a.

```
Enter matrix size: 4
Enter beam size: 2
Enter initial state 1: 1 2 3 4
Enter initial state 2: 2 1 3 4
```

```
Level 1 successor:
2 1 3 1 (conflicts: 2)
2 1 1 4 (conflicts: 2)
```

```
Level 2 successor:
2 4 3 1 (conflicts: 1)
2 4 1 4 (conflicts: 1)
```

```
Level 3 successor:
2 4 1 3 (conflicts: 0)
```

```
Solution found!
```

3b.

```
PS D:\Devops> javac MapColoring.java
PS D:\Devops> java MapColoring
Enter the number of states: 13
Enter the number of colors: 4
Enter the colors: R G B Y
Enter the connections (-1 for end):
1 2
2 3
3 4
1 5
2 6
3 7
4 8
5 6
6 7
7 8
5 9
6 10
7 11
8 12
9 10
10 11
11 12
11 13
-1
```

Output:

```
1 - R
2 - G
3 - R
4 - G
5 - G
6 - R
7 - G
8 - R
9 - R
10 - G
11 - R
12 - G
13 - G
```

3c.

```
PS D:\Devops> javac AlphaBeta.java
Note: Recompile with -Xlint:unchecked for details.
PS D:\Devops> java AlphaBeta
Enter the number of Nodes: 9
Enter the connections (-1 for end):
1 2
1 3
1 4
2 5
2 6
3 7
3 8
4 9
-1
Enter utility values for leaf nodes:
Node 5: 5
Node 6: 10
Node 7: -3
Node 8: -4
Node 9: 2

Best Move for MAX: Node 1 -> Node 2
MAX Val: 5
```